

LegalLens AI

AI-Powered Legal Assistance & Access

Project Design, Architecture, Features, Security, Efficiency, Testing and Accessibility Plan

Core idea:

An evidence-grounded legal document assistant that helps people understand, compare and navigate legal documents in plain language, with clause-level citations, risk indicators and actionable next steps.

Important: The system provides legal information and document assistance. It is not intended to replace a qualified legal professional.

1. Project Concept

One-line pitch: An AI-powered legal document assistant that helps people understand, compare, and navigate legal documents in plain language, with evidence-backed answers, clause-level citations, risk detection, and actionable checklists.

Core Workflow

```
User
↓
PDF / DOCX / Text / Question / Comparison
↓
Document Processing
■ Text extraction
■ OCR for scanned documents
■ Page and clause mapping
↓
Legal Knowledge Layer
■ Chunking
■ Metadata
■ Embeddings / Search
↓
AI Legal Reasoning
■ RAG
■ Rules
■ Safety / guardrails
↓
Evidence Validator
■ Citation and grounding verification
↓
User Output
■ Summary
■ Risks / important clauses
■ Comparison
■ Checklist
■ Questions / next steps
```

2. Core Features

2.1 Understand My Document

Users can upload employment contracts, rental agreements, NDAs, service agreements, loan agreements, insurance documents, terms and conditions, policies, notices and similar documents.

- Plain-language document summary.
- Identify parties, dates, duration and major obligations.
- Highlight important clauses such as termination, confidentiality, IP and dispute resolution.
- Generate questions the user may want to discuss with a legal professional.
- Attach source references such as page and clause numbers.

2.2 Ask Questions About the Document

```
User: "Can my employer terminate me immediately?"

System:
1. Retrieve relevant clauses.
2. Analyze only the retrieved evidence.
3. Produce an explanation.
4. Show page / clause citations.
5. State when the document does not provide enough evidence.
```

Example output: The uploaded agreement describes termination conditions in Clause 9.2 and a notice requirement in Clause 9.3. The response is an explanation of the uploaded document, not legal advice.

2.3 Contract Comparison

Allow users to upload Contract A and Contract B and produce a side-by-side comparison.

Category	Contract A	Contract B
Duration	1 year	2 years
Notice period	30 days	60 days
Termination	Clause 8	Clause 9
Payment	■50,000	■55,000
Confidentiality	Yes	Yes
IP ownership	Employer	Joint
Dispute resolution	Court	Arbitration

The system should present differences and sources without telling the user which contract to choose.

2.4 Clause and Risk Scanner

- Termination
- Payment and penalties
- Renewal and automatic renewal
- Liability and indemnity
- Confidentiality
- Intellectual property
- Restrictive clauses
- Dispute resolution and arbitration
- Jurisdiction
- Data and privacy
- Notice period
- Refund and cancellation

Use language such as “**may warrant professional review**” rather than making unsupported claims that a clause is illegal.

2.5 Next Steps and Lawyer Question Checklist

- 60-day notice period
- Confidentiality obligation
- IP assignment
- Restrictive covenant

- Read the relevant clause completely.
- Confirm how the notice period applies.
- Ask the other party whether the clause is negotiable.
- Save a copy of the signed agreement.
- Consider professional legal review.

1. How does the restrictive clause apply to my situation?
2. What happens if I resign before the notice period ends?
3. What obligations survive termination?

[illegible]

4. AI Provider Architecture

```
LLM_PROVIDER=gemini
GEMINI_API_KEY=...

# OR

LLM_PROVIDER=groq
GROQ_API_KEY=...
```

```

class LLMProvider:
    async def generate(self, prompt: str) -> str:
        raise NotImplementedError

class GeminiProvider(LLMProvider):
    ...

class GroqProvider(LLMProvider):
    ...

```

This abstraction improves maintainability, testing and provider flexibility.

5. Retrieval-Augmented Generation (RAG)

```

USER QUESTION
↓
Query Analyzer
↓
Question classification
↓
Relevant document retrieval
↓
Top chunks / clauses
↓
Evidence builder
↓
LLM generation
↓
Citation validator
↓
Final answer

```

The document is not sent in full for every question. Relevant chunks are retrieved using embeddings and metadata, reducing token usage, latency and hallucination opportunities.

6. Hallucination and Grounding Controls

A central safety rule is: **if an answer cannot be supported by retrieved document evidence, do not present it as a document fact.**

```

Possible response states:

DOCUMENT_SUPPORTED
GENERAL_INFORMATION
INSUFFICIENT_EVIDENCE
PROFESSIONAL_REVIEW_RECOMMENDED

```

Example citation structure:

```

{
  "answer": "...",
  "sources": [
    {
      "document": "employment.pdf",
      "page": 5,
      "clause": "9.2"
    }
  ],
  "confidence": "document_supported"
}

```

7. Security Design

- **Upload validation:** verify extension, MIME type, file signature, file size and page count.

- **Malware protection:** scan uploaded files before processing.
- **Private file access:** never expose public file paths; require authenticated and authorized access.
- **Encryption:** use HTTPS and encrypted storage where supported.
- **Authentication:** secure sessions or JWT, strong password hashing and rate limiting.
- **Authorization:** verify that the requesting user owns or can access the document.
- **Secrets:** keep API keys and database credentials in environment variables, never in source code.

Prompt Injection Protection

Treat uploaded documents as untrusted data, not as instructions. Retrieved document text must never override system or application instructions.

```
SYSTEM INSTRUCTIONS
↓
APPLICATION INSTRUCTIONS
↓
RETRIEVED DOCUMENT (UNTRUSTED EVIDENCE)
↓
USER QUESTION
```

8. Privacy Design

- Documents are private to the user's account by default.
- Provide document and conversation deletion.
- Minimize the amount of personal information sent to the LLM.
- Retrieve only relevant document sections where possible.
- Provide a visible privacy explanation in the product.
- Do not log raw legal documents unnecessarily.

```
Privacy Center
✓ Documents encrypted
✓ Documents private to your account
✓ No document sharing by default
✓ Delete document
✓ Delete conversation
✓ Export data
```

9. Efficiency

- Use asynchronous document processing for large files.
- Pre-compute embeddings once during indexing.
- Retrieve only the most relevant chunks for each query.
- Use metadata filtering before vector retrieval where appropriate.
- Cache reusable results where safe.
- Limit LLM context to necessary evidence.
- Measure latency and token usage during evaluation.

```
100 document chunks
↓
Similarity search
↓
Top 5
↓
Optional reranking
↓
Top 3
↓
LLM
```

10. Code Quality and Repository Structure

```
legallens/
  frontend/
  app/
  components/
  hooks/
  lib/
  types/
  tests/
  backend/
  app/
  api/routes/
  core/
  models/
  schemas/
  services/
  ai/
  main.py
  tests/
  database/
  migrations/
  seed/
  docs/
  architecture.md
  security.md
  api.md
  threat-model.md
  .env.example
  docker-compose.yml
  README.md
  LICENSE
```

Use TypeScript types, Python type hints, formatting/linting, clear service boundaries, small reusable functions, centralized configuration, structured logging and documented APIs.

11. Database Design

Table	Important fields
users	id, email, password_hash, created_at
documents	id, user_id, filename, document_type, status, created_at
document_pages	id, document_id, page_number, text
document_chunks	id, document_id, page_number, section, content, embedding
conversations	id, user_id, document_id, created_at
messages	id, conversation_id, role, content, created_at
citations	id, message_id, document_id, page_number, clause, quoted_text
analyses	id, document_id, analysis_type, result, created_at

Table	Important fields
comparison_reports	id, user_id, document_a, document_b, result, created_at

12. Testing Strategy

```
tests/
  unit/
  integration/
  security/
  rag/
  evaluation/
```

- Unit tests: PDF extraction, chunking, citation extraction, risk classification, authentication and authorization.
- Integration tests: upload → process → index → ask → retrieve → generate → validate citation.
- Security tests: unauthorized access, path traversal, malicious uploads, oversized files, invalid tokens, rate limits, prompt injection, XSS and SQL injection.
- RAG evaluation: measure retrieval accuracy, citation accuracy, groundedness, hallucination rate and latency.

Create a labeled evaluation dataset, for example 50–100 document questions, with expected page and clause references. Only publish evaluation metrics that were actually measured.

13. Accessibility

- Keyboard navigation.
- Screen-reader friendly semantic HTML.
- Visible focus states.
- Accessible labels and forms.
- Sufficient contrast.
- Responsive mobile interface.
- Scalable text.
- Plain-language explanations.
- Multilingual support where feasible.
- Optional voice input/output for accessibility.

Plain-Language Mode

```
Original:
"The lessee shall indemnify..."

Simple explanation:
"You may have to compensate the other party if
certain losses occur."

[View original clause]
```

Keep the original legal text available so users can verify the simplified explanation.

14. Evidence View

Every important AI claim should be traceable to the original document.

Phase	Deliverables
1. Foundation	Next.js, FastAPI, PostgreSQL, authentication, Docker, environment configuration
2. Documents	PDF/DOCX upload, OCR, extraction, page mapping and storage
3. RAG	Chunking, embeddings, pgvector, retrieval and citations
4. AI	Gemini/Groq abstraction, Q&A, summaries and plain-language explanations
5. Legal intelligence	Clause detection, obligations, risk indicators, inconsistencies and checklists
6. Comparison	A/B comparison, clause alignment and difference reports
7. Security	Authorization, upload validation, rate limits, prompt-injection protection
8. Quality	Tests, evaluation dataset, CI/CD, logging and monitoring
9. Accessibility	Keyboard support, screen readers, responsive UI, multilingual and plain-language modes

19. Feature-to-Scoring Alignment

uation field	Implementation focus
e Quality — High Impact	Modular architecture, TypeScript/Python typing, clean APIs, service layer, linting, formatting and documentation
urity — Medium Impact	Authentication, authorization, secure uploads, encrypted storage, rate limiting, secret management and prompt-injection defe
ency — Medium Impact	RAG, embeddings, retrieval, caching, async processing and controlled LLM context
ng — Low Impact	Unit, integration, security and RAG evaluation tests plus CI
ssibility — Low Impact	Keyboard navigation, screen readers, responsive design, plain language, multilingual and voice support
lem Statement Alignment — High Impact	Document simplification, Q&A, comparison, clause analysis, risks, summaries, checklists and next steps

20. Recommended Final Demo

```
Scenario:
Upload Employment_Agreement_A.pdf
Upload Employment_Agreement_B.pdf

Step 1:
"Explain this contract to me like I'm not a lawyer."
→ Plain-language summary

Step 2:
"What obligations do I have?"
→ Obligations + citations

Step 3:
"Find clauses I should pay attention to."
→ Clause analysis

Step 4:
"Compare this with my second contract."
→ Side-by-side differences

Step 5:
"What questions should I ask a lawyer?"
→ Personalized question checklist

Step 6:
"View Source"
→ Original PDF page + highlighted evidence
```

This demonstration directly covers understanding, comparison, important clauses, questions, actionable outputs and evidence-backed assistance.

21. What Not to Build

- Do not make the product only a generic chat interface connected to an LLM.
- Do not market it as an AI lawyer or replacement for legal professionals.
- Do not allow unsupported legal claims to appear as facts.
- Do not hard-code a single legal jurisdiction as universally applicable.
- Do not expose uploaded documents publicly.
- Do not place API keys in source code or commit .env files.
- Do not fabricate AI evaluation metrics.

22. Final Project Positioning

Recommended project statement:

Legallens AI is an evidence-grounded legal document assistant designed to improve access to legal information. It helps ordinary users understand their own legal documents, identify important clauses and obligations, compare documents, verify AI explanations against source evidence, and prepare actionable checklists and questions for a qualified legal professional.

The system provides information and document assistance rather than replacing professional legal advice.

Core Differentiator

The strongest version of the project is not simply an AI that “knows law.” It is an **evidence-grounded system** that helps people understand their own legal documents, see important differences and clauses, verify AI explanations against the original document, and prepare informed next steps or questions for a legal professional.

23. Submission Checklist

- Working frontend and backend.
- Secure document upload and private storage.
- Document extraction with page mapping.
- RAG retrieval with citations.
- Gemini/Groq provider abstraction.
- Plain-language summary.
- Document Q&A.;
- Clause and obligation extraction.
- Risk/attention indicators.
- A/B document comparison.
- Action checklist and lawyer-question generator.
- Evidence/source viewer.
- Legal information notice.

- Authentication and authorization.
- Security tests.
- Unit and integration tests.
- RAG evaluation dataset.
- Accessibility checks.
- README and architecture documentation.
- .env.example with no real secrets.
- Docker/deployment instructions.

Target outcome: A polished, secure, testable and accessible GenAI application that demonstrates clear alignment with the legal-assistance problem while maintaining human oversight and professional-review boundaries.